

AI TRAINING – NUMPY ASSIGNMENT

Arrays, Dimensions, Indexing, Slicing & Vectorized Operations

Learn → Create Arrays → Access Data → Process Efficiently

1. What You Will Learn

- Create NumPy arrays from Python lists.
- Understand 1D, 2D, and 3D array dimensions and the shape attribute.
- Access array elements using positive and negative indexing.
- Select portions of arrays using slicing.
- Perform vectorized arithmetic and comparison operations without writing loops for each element.
- Complete a small marks-analysis task using NumPy.

2. Student Deliverable – Keep It Simple

Create one Python script named: `numpy_assignment_YourName.py`

Use NumPy (import numpy as np). Keep the code readable and divide the main task into functions where useful.

- Create and inspect arrays.
- Demonstrate dimensions and shape.
- Use indexing and slicing to access data.
- Apply vectorized operations.
- Print clear results and test your code.

3. Part A – Create NumPy Arrays

Import NumPy and complete these tasks:

1. Create a 1D array from [10, 20, 30, 40, 50].
2. Create a 2D array from [[10, 20, 30], [40, 50, 60]].
3. Create a 3D array using the values 1 through 8, arranged into two 2×2 blocks.
4. Print each array and identify its data type using dtype.
5. Create an array of five zeros and an array of five ones.

CODE:

```
import numpy as np
arr1 = np.array([10, 20, 30, 40, 50])
arr2 = np.array([[10, 20, 30],
                 [40, 50, 60]])
arr3 = np.array([[[1, 2],
                  [3, 4]],
                 [[5, 6],
                  [7, 8]]])
```

```
print("1D Array:")
print(arr1)
print("Data type:", arr1.dtype)
print("\n2D Array:")
print(arr2)
print("Data type:", arr2.dtype)
print("\n3D Array:")
print(arr3)
print("Data type:", arr3.dtype)
zeros = np.zeros(5)
ones = np.ones(5)
print("\nArray of zeros:")
print(zeros)
print("Zeros dtype:", zeros.dtype)
print("\nArray of ones:")
print(ones)
print("Ones dtype:", ones.dtype)
```

OUTPUT:

1D Array:

[10 20 30 40 50]

Data type: int64

2D Array:

[[10 20 30]

[40 50 60]]

Data type: int64

3D Array:

[[[1 2]

[3 4]]

[[5 6]

[7 8]]]

Data type: int64

Array of zeros:

[0. 0. 0. 0. 0.]

Zeros dtype: float64

Array of ones:

[1. 1. 1. 1. 1.]

Ones dtype: float64

4. Part B – Dimensions and Shape

For each array created in Part A:

1. Print ndim to identify the number of dimensions.
2. Print shape to show the size along each dimension.
3. Print size to count the total number of elements.
4. In your own words, explain the difference between ndim, shape, and size.
5. Predict the shape of a 3-row, 4-column array before running the code.

CODE:

```
import numpy as np
arr1 = np.array([10, 20, 30, 40, 50])
arr2 = np.array([[10, 20, 30],
                 [40, 50, 60]])
arr3 = np.array([[[1, 2],
                  [3, 4],
                  [5, 6],
                  [7, 8]]])

print("1D Array")
print("ndim:", arr1.ndim)
print("shape:", arr1.shape)
print("size:", arr1.size)
print("\n2D Array")
print("ndim:", arr2.ndim)
print("shape:", arr2.shape)
print("size:", arr2.size)
print("\n3D Array")
print("ndim:", arr3.ndim)
print("shape:", arr3.shape)
print("size:", arr3.size)
print("\nZeros Array")
print("ndim:", zeros.ndim)
print("shape:", zeros.shape)
print("size:", zeros.size)
print("\nOnes Array")
print("ndim:", ones.ndim)
print("shape:", ones.shape)
print("size:", ones.size)

print("Predicted shape:", (3, 4))
```

OUTPUT:

1D Array

ndim: 1
shape: (5,)
size: 5
2D Array
ndim: 2
shape: (2, 3)
size: 6
3D Array
ndim: 3
shape: (2, 2, 2)
size: 8
Zeros Array
ndim: 1
shape: (5,)
size: 5
Ones Array
ndim: 1
shape: (5,)
size: 5
Predicted shape: (3, 4)

5. Part C – Indexing

Use this array: marks = np.array([72, 85, 91, 66, 78, 95])

1. Print the first element and the last element.
2. Print the element at index 2.
3. Print the second-last element using negative indexing.
4. Change the value 66 to 70 and print the updated array.
5. Use a 2D array [[11, 12, 13], [21, 22, 23]] to access 22 and 13.

CODE:

```
import numpy as np
marks = np.array([72, 85, 91, 66, 78, 95])
print("First element:", marks[0])
print("Last element:", marks[-1])
print("Element at index 2:", marks[2])
print("Second-last element:", marks[-2])
marks[3] = 70
print("Updated array:", marks)
arr = np.array([[11, 12, 13],
                [21, 22, 23]])
```

```
print("Element 22:", arr[1, 1])
print("Element 13:", arr[0, 2])
OUTPUT:
First element: 72
Last element: 95
Element at index 2: 91
Second-last element: 78
Updated array: [72 85 91 70 78 95]
Element 22: 22
Element 13: 13
```

6. Part D – Slicing

Use this array: `values = np.array([5, 10, 15, 20, 25, 30, 35, 40])`

1. Select the first four elements.
2. Select the elements from index 2 up to, but not including, index 6.
3. Select every second element.
4. Select the last three elements.
5. For `matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])`, select the first two rows and the last two columns.
6. Write one sentence explaining why the stop index is not included in a slice.

CODE:

```
import numpy as np
values = np.array([5, 10, 15, 20, 25, 30, 35, 40])
print("First four elements:", values[:4])
print("Index 2 to 5:", values[2:6])
print("Every second element:", values[::2])
print("Last three elements:", values[-3:])
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("First two rows and last two columns:")
print(matrix[:2, -2:])
print("The stop index is excluded because Python slicing")
print("includes the start index but stops before the stop index.")
```

OUTPUT:

```
First four elements: [ 5 10 15 20]
Index 2 to 5: [15 20 25 30]
Every second element: [ 5 15 25 35]
Last three elements: [30 35 40]
```

First two rows and last two columns:

```
[[2 3]
 [5 6]]
```

The stop index is excluded because Python slicing includes the start index but stops before the stop index.

7. Part E – Vectorized Operations

Use these arrays:

```
a = np.array([10, 20, 30, 40])
```

```
b = np.array([1, 2, 3, 4])
```

1. Calculate $a + b$, $a - b$, $a * b$, and a / b .
2. Add 5 to every element of a without using a loop.
3. Multiply every element of a by 3.
4. Find the elements of a that are greater than 20 using a comparison operation.
5. Use a Boolean condition to select marks greater than or equal to 80 from `np.array([55, 82, 91, 67, 88])`.
6. In your own words, explain what vectorized operations do.

CODE:

```
import numpy as np
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Add 5:", a + 5)
print("Multiply by 3:", a * 3)
print("Greater than 20:", a[a > 20])
marks = np.array([55, 82, 91, 67, 88])
result = marks[marks >= 80]
print("Marks >= 80:", result)
```

OUTPUT:

```
Addition: [11 22 33 44]
Subtraction: [ 9 18 27 36]
Multiplication: [ 10 40 90 160]
Division: [10. 10. 10. 10.]
Add 5: [15 25 35 45]
Multiply by 3: [ 30 60 90 120]
Greater than 20: [30 40]
Marks >= 80: [82 91 88]
```

8. Part F – Debugging Practice

For each snippet, identify the problem and correct it.

Debugging Problem 1

```
import numpy as np
arr = np.array([10, 20, 30])
print(arr[3])

CODE:import numpy as np
arr = np.array([10, 20, 30])
print(arr[2])

OUTPUT:30
```

Explanation: `arr[3]` causes an `IndexError` because valid indices are 0, 1, and 2. Use `arr[2]` to access the last element.

Debugging Problem 2

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[1:10])

CODE:import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[1:])

OUTPUT:[2 3 4]
```

Explanation: `arr[1:10]` is valid Python. Slicing safely stops at the end of the array, so it returns `[2 3 4]`. The original code does not actually fail.

Debugging Problem 3

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20])
print(a + b)

CODE:import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)

OUTPUT:[11 22 33]
```

Explanation: The arrays have incompatible shapes (3,) and (2,). Make both arrays the same length to perform element-wise addition.

Debugging Problem 4

```
import numpy as np
arr = np.array([[1, 2], [3, 4]])
print(arr[2, 0])

CODE:import numpy as np
arr = np.array([[1, 2], [3, 4]])
print(arr[1, 0])

OUTPUT:3
```

Explanation: `arr[2, 0]` causes an `IndexError` because the array has only two rows. Use `arr[1, 0]` to access the value 3.

9. Part G – Main Hands-on Activity: Student Marks Analyzer

Create a NumPy-based program to analyze student marks. Use this data:

```
marks = np.array([[78, 85, 92], [66, 74, 81], [90, 88, 95], [55, 69, 72]])
```

Each row represents one student; each column represents a subject.

Your program should:

- Print the full marks array, its dimensions, and its shape.
- Print the marks of the first student.
- Print the marks for the second subject for all students.
- Add 5 bonus marks to every score using a vectorized operation.
- Cap adjusted marks at 100 using `np.minimum` or an equivalent NumPy operation.
- Calculate each student's average marks using an axis argument.
- Find the highest mark in the full array.
- Select all adjusted scores that are 80 or above.
- Print results with meaningful labels.

CODE:

```
import numpy as np
marks = np.array([
    [78, 85, 92],
    [66, 74, 81],
    [90, 88, 95],
    [55, 69, 72]
])
print("Marks Array:\n", marks)
print("Dimensions:", marks.ndim)
print("Shape:", marks.shape)
print("First Student Marks:", marks[0])
print("Second Subject Marks:", marks[:, 1])
adjusted_marks = np.minimum(marks + 5, 100)
print("Adjusted Marks:\n", adjusted_marks)
averages = np.mean(adjusted_marks, axis=1)
print("Student Averages:", averages)
print("Highest Mark:", np.max(marks))
print("Adjusted Scores >= 80:", adjusted_marks[adjusted_marks >= 80])
print("Highest Adjusted Mark:", np.max(adjusted_marks))
```

OUTPUT:

Marks Array:

```
[[78 85 92]
```

```
[66 74 81]
```

[90 88 95]

[55 69 72]]

Dimensions: 2

Shape: (4, 3)

First Student Marks: [78 85 92]

Second Subject Marks: [85 74 88 69]

Adjusted Marks:

[[83 90 97]

[71 79 86]

[95 93 100]

[60 74 77]]

Student Averages: [90. 78.66666667 96. 70.33333333]

Highest Mark: 95

Adjusted Scores >= 80: [83 90 97 86 95 93 100]

Highest Adjusted Mark: 100

10. Part H – Explain in Your Own Words

1. What is NumPy, and why is it useful?

Answer: NumPy (Numerical Python) is a Python library used to work with arrays and perform mathematical calculations efficiently.

2. What is a NumPy array?

Answer: A NumPy array is a data structure used to store multiple elements in one or more dimensions.

3. What is the difference between a Python list and a NumPy array?

Answer: A Python list stores multiple items and can contain different data types, while a NumPy array stores elements efficiently and supports fast mathematical operations.

4. What do ndim, shape, size, and dtype represent?

Answer: ndim: Number of dimensions.

shape: Size along each dimension.

size: Total number of elements.

dtype: Data type of the elements.

5. What is indexing? What is negative indexing?

Answer: Indexing: Indexing is used to access an element in an array using its position, starting from index 0.

Negative Indexing: Negative indexing is used to access elements in an array from the end, starting with index -1.

6. What is slicing?

Answer: Slicing selects a range of elements from an array using start, stop, and step values.

7. What is a vectorized operation?

Answer: A vectorized operation performs an operation on all elements of a NumPy array at once without using an explicit loop.

8. Why must arrays generally have compatible shapes for element-wise arithmetic?

Answer: Arrays must have compatible shapes for element-wise arithmetic so that their elements can be matched correctly for calculations.

11. Daily Deliverable

Submit your working Python script: `numpy_assignment_YourName.py`

Your script should demonstrate:

- Array creation and dtype
- 1D, 2D, and 3D dimensions
- ndim, shape, and size
- Positive and negative indexing
- 1D and 2D slicing
- Vectorized arithmetic and comparisons
- A complete marks-analysis activity
- Readable code and clear output

12. Student Submission Checklist

- ☐ I can create NumPy arrays.✓
- ☐ I can identify 1D, 2D, and 3D arrays.✓
- ☐ I understand ndim, shape, size, and dtype.✓
- ☐ I can access elements using indexing.✓
- ☐ I can extract data using slicing.✓
- ☐ I can perform arithmetic without element-by-element loops.✓
- ☐ I can filter array values using Boolean conditions.✓
- ☐ I tested and corrected the debugging problems.✓
- ☐ I can explain the main concepts in my own words.✓

13. After This Assignment, You Should Be Able To

- Create and inspect NumPy arrays.
- Navigate 1D, 2D, and 3D array data.
- Select elements and subarrays using indexing and slicing.
- Process numerical data using vectorized operations.
- Use NumPy to perform a small practical data-analysis task.

Key Principle: Use NumPy to organize numerical data and process many values efficiently.